Statistics and Data Science

LEPL1109: Data science part

L. Jacques

2.4. Resampling Methods

2.5. More on Classification Methods

Given a model $f_{\beta,\gamma}$ (or non-parametric f_γ) and an SL algorithm (or learner)

$$\mathcal{A}: \quad \underbrace{\{(x_i, y_i)\}_{i=1}^N}_{\mathcal{T}}, \underbrace{\{f_{\beta,\gamma} : \beta \in \mathcal{B}\}}_{\mathcal{F}_\gamma} \quad \rightarrow \quad f_{\hat{\beta},\gamma}(\cdot)$$

i.e., $\mathcal{A}$ learns (fits) a model $f_{\hat{\beta},\gamma}$ from a dataset $\mathcal{T}$.

1. **Fitting a parametric model** $\equiv$ minimizing the empirical risk $\hat{\mathrm{R}}_{\mathcal{T}}(f)$ over $\mathcal{T}$:

$$f_{\hat{\beta},\gamma} = \mathcal{A}(\mathcal{T}, \mathcal{F}_\gamma) := \arg\min_{f \in \mathcal{F}_\gamma} \underbrace{\frac{1}{N}\sum_{i=1}^N \ell(y_i, f(x_i))}_{\hat{\mathrm{R}}_{\mathcal{T}}(f)},$$

or equivalently, finding

$$\hat{\beta} := \arg\min_\beta \frac{1}{N}\sum_{i=1}^N \ell(y_i, f_{\beta,\gamma}(x_i)).$$

2. **Fitting a nonparametric model** $\equiv$ defining it from $\mathcal{T}$ (as for k-NN)
$\Rightarrow$ no optimization (example, k-NN)

Generalization error: given a new, independent dataset $\mathcal{T}' := \{(x'_j, y'_j)\}_{j=1}^{N'}$, how can we bound the error on $\mathcal{T}'$? The error

$$\hat{\mathrm{R}}_{\mathcal{T}'}(f_{\hat{\beta},\gamma}) := \frac{1}{N'} \sum_{j=1}^{N} \ell(y'_j, f_{\hat{\beta},\gamma}(x'_j)).$$

$\Rightarrow$ Do the model $f_{\hat{\beta},\gamma}$ generalizes well?

Challenges:

- New dataset $\mathcal{T}'$ not always available.
- How to estimate the hyperparameter γ?
- In general, how to evaluate if there is a better model $\mathcal{F}'$ than $\mathcal{F}$?

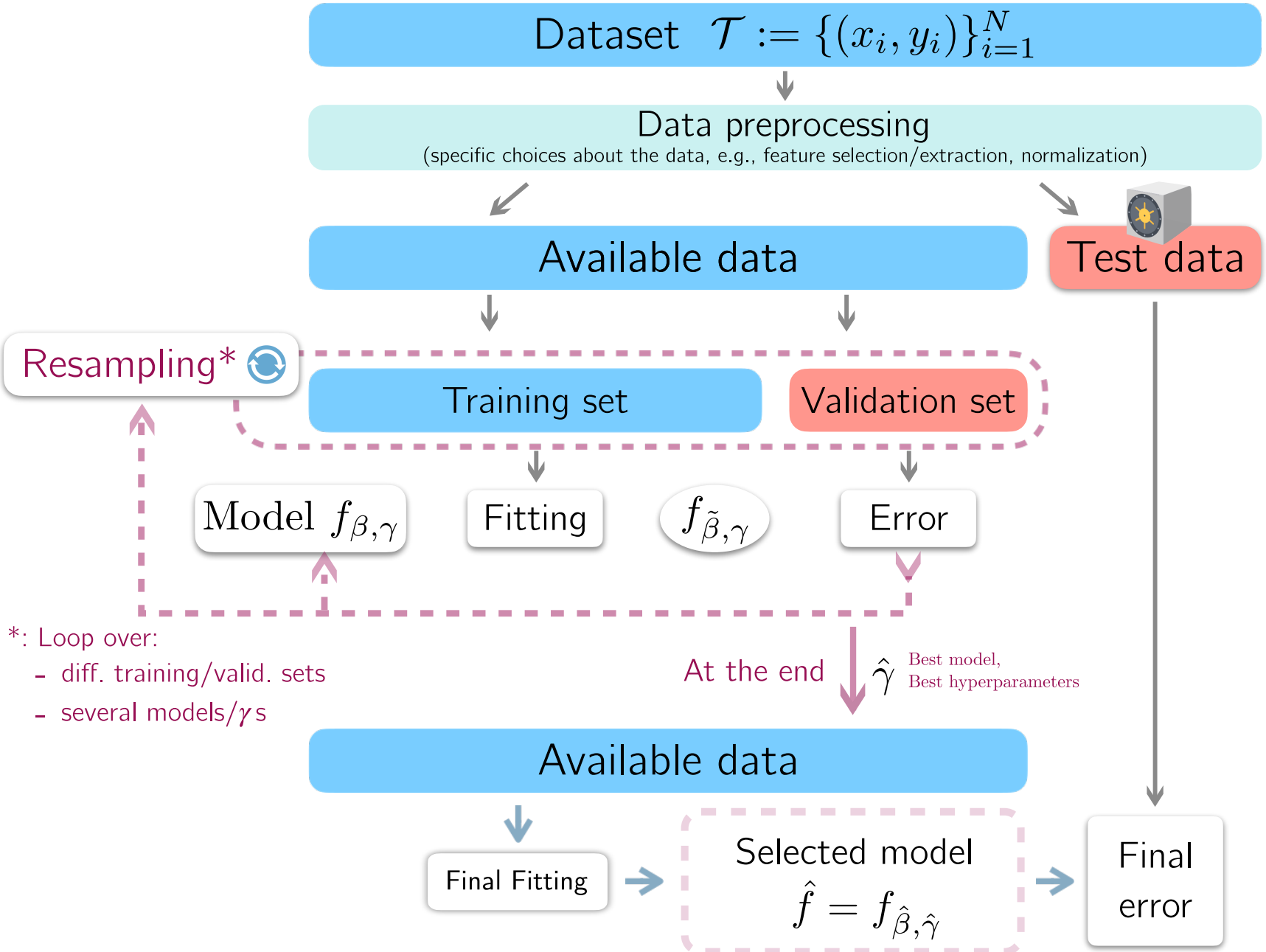
Solution: Use resampling methods $\equiv$ simulate $\mathcal{T}'$ from $\mathcal{T}$

Resampling methods are repeatedly

- *(i)* drawing samples (at random) from a training set,
- *(ii)* refitting a model of interest on the set of these samples.

Why resampling? obtain additional information about the fitted model

- for model assessment: evaluate model's performance (*e.g.*, test error), *e.g.*, estimate the *variability* of the linear model $f_\beta(x) := x^\top \beta$ $(x, \beta \in \mathbb{R}^p)$
- for model selection: select (hyper)parameters (*e.g.*, p in LS, k in k-NN), or improve the feature space (*e.g.*, remove some X_i's, add others).


Dataset $\mathcal{T} := \{(x_i, y_i)\}_{i=1}^{N}$
Data preprocessing
(specific choices about the data, e.g., feature selection/extraction, normalization)
Available data
Test data
Resampling*
Training set
Validation set
Model $f_{\beta,\gamma}$
Fitting
$f_{\tilde{\beta},\gamma}$
Error
*: Loop over:
- diff. training/valid. sets
- several models/γs
At the end
$\hat{\gamma}$
Best model,
Best hyperparameters
Available data
Final Fitting
Selected model
$\hat{f} = f_{\hat{\beta},\hat{\gamma}}$
Final error

Resampling principles (3/3)

Pros & **cons** of resampling methods:

- 👍 **Pros**: Access information not available in a single fit.
- 👎 **Cons**: Could be computationally intensive
  (but $\exists$ strategies to mitigate this drawback).

We consider only **cross validation methods**:

- Validation Set Approach (covered)
- Leave-One-Out Cross-Validation (LOOCV) (covered)
- K-Fold Cross-Validation (K-Fold CV) (covered)
- (Bootstrap can be used too) (not covered, see DH's part)

Remark: All of these are implemented in scikit learn

Ideal objective in supervised learning:

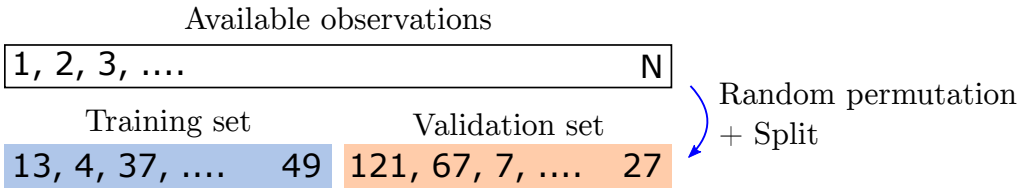
- Reduce the error of a machine learning methods on *new observations*, *i.e.*, the test error (*i.e.*, the risk, or the empirical risk on test data).

However, new observations, or test dataset, not often available.

One possibility:

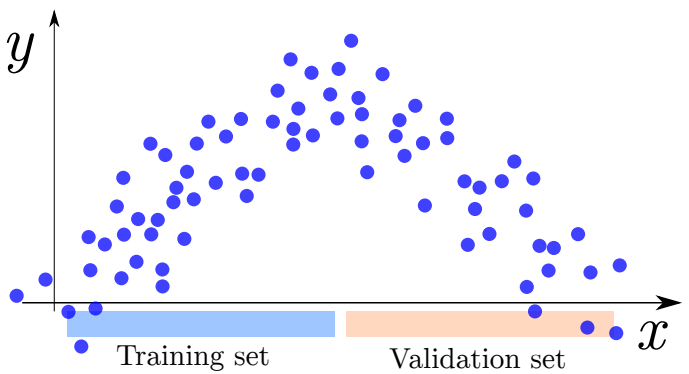
- holding out a subset of the training data from the fitting process.
- applying the fitted ML method to the held out data.

Randomly divide available data into *training* set and a *validation* set.



$\Rightarrow$ Estimate *test error* on the validation set.

❓ **Question**: Why randomizing is useful? Imagine this situation



for instance, time series ($x \in \mathbb{R}$), image pixels ($x \in \mathbb{R}^2$).

How LS or k-NN will behave on this?

You must break correlation/dependence in $\{x_i\}$'s $\Rightarrow$ randomize the sets

Mathematically: For a 50%/50% split (to get the idea),

1. Given an (even) number N of observations $\{(x_i, y_i)\}_{i=1}^N$, a model $\hat{f}$, and a *random permutation* $\pi : [N] \to [N]$, we form

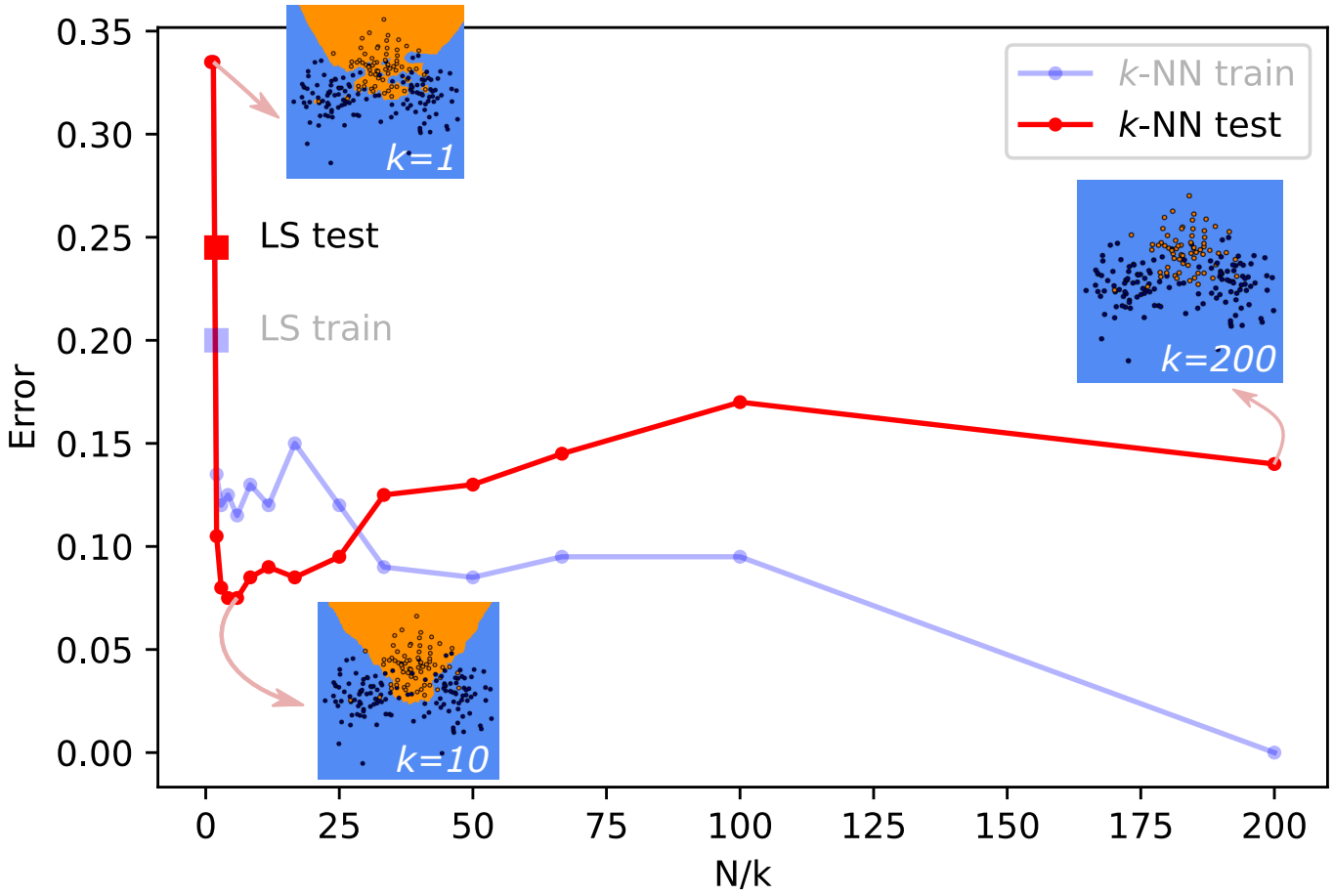
- Training set: $\mathcal{T}_{\text{tr}} = \{(x_{\pi(i)}, y_{\pi(i)})\}_{i=1}^{N/2}$ *e.g.*, Training set: 13, 4, 37, ... 49
- Validation set: $\mathcal{T}_{\text{v}} = \{(x_{\pi(i)}, y_{\pi(i)})\}_{i=N/2+1}^{N}$ *e.g.*, Validation set: 121, 67, 7, ... 27

2. We fit $\hat{f}$ on $\mathcal{T}_{\text{tr}}$ and compute the *validation error*

$$\text{MSE}_{\text{v}} := \text{MSE}(\mathcal{T}_{\text{v}}) = \frac{2}{N} \sum_{i=(N/2)+1}^{N} \left(y_{\pi(i)} - \hat{f}(x_{\pi(i)})\right)^2, \qquad \text{(for regression)}$$

$$\text{Err}_{\text{v}} := \text{Err}(\mathcal{T}_{\text{v}}) = \frac{2}{N} \sum_{i=(N/2)+1}^{N} I\left(y_{\pi(i)} \neq \hat{f}(x_{\pi(i)})\right). \qquad \text{(for classification)}$$

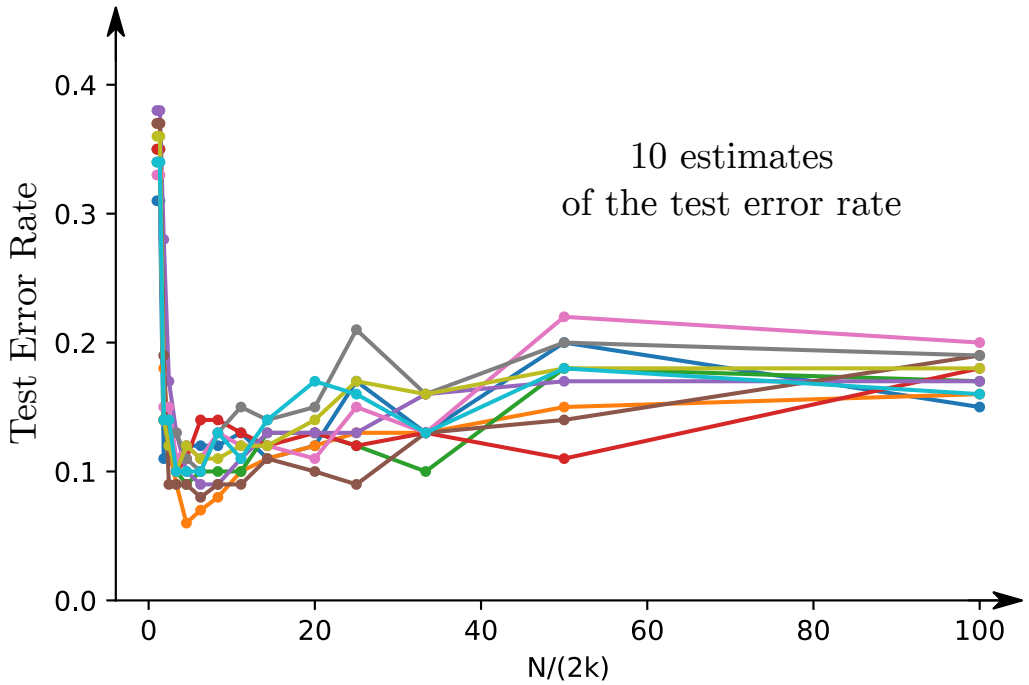
Remark: This was how I estimated the test error earlier:



Training error of LS and k-NN classifiers

Drawbacks for the validation set approach:

- Variability of the estimate of the test error rate
  e.g., same experiments on 10 possible random splits



- The training set uses only half of the observation
  $\Rightarrow$ possible *over-estimation* of the test error

Leave-One-Out Cross-Validation (1/4)

LOOCV: Leverage the validation set approach with

- Size of training set $= N - 1$
- Size of validation set $= 1$ (single observation)

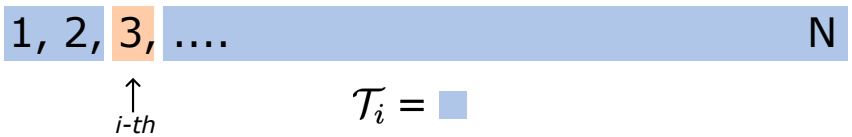
There are thus N possible configurations (or splits) $\rightarrow$ use them all!

$\Rightarrow$ After the *available/test* data plit, split *available* data as:



Mathematically: For each i in $\{1, \cdots, N\}$,

1. Define $\mathcal{T}_i := \{(x_j, y_j) : j \in [N],\ j \neq i\}$ for $1 \leqslant i \leqslant N$,



2. Fit your model on $\mathcal{T}_i := \{(x_j, y_j) : j \in [N],\ j \neq i\}$ and get $\hat{f}_i$.
3. Compute the prediction error (with sum $\equiv$ single element)

$$\mathrm{MSE}_i := \mathrm{MSE}(\{(x_i, y_i)\}) = \left(y_i - \hat{f}_i(x_i)\right)^2, \qquad \text{(for regression)}$$

$$\mathrm{Err}_i := \mathrm{Err}(\{(x_i, y_i)\}) = I\left(y_i \neq \hat{f}_i(x_i)\right). \qquad \text{(for classification)}$$

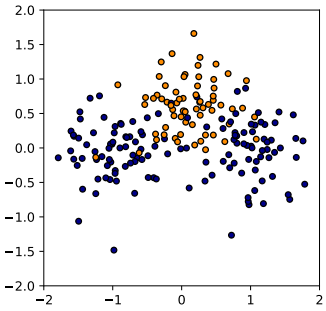
In LOOCV, test error estimation = average of all prediction errors

$$\mathrm{CV}_{(N)} = \frac{1}{N} \sum_{i=1}^{N} \mathrm{MSE}_i, \qquad \text{(for regression)}$$

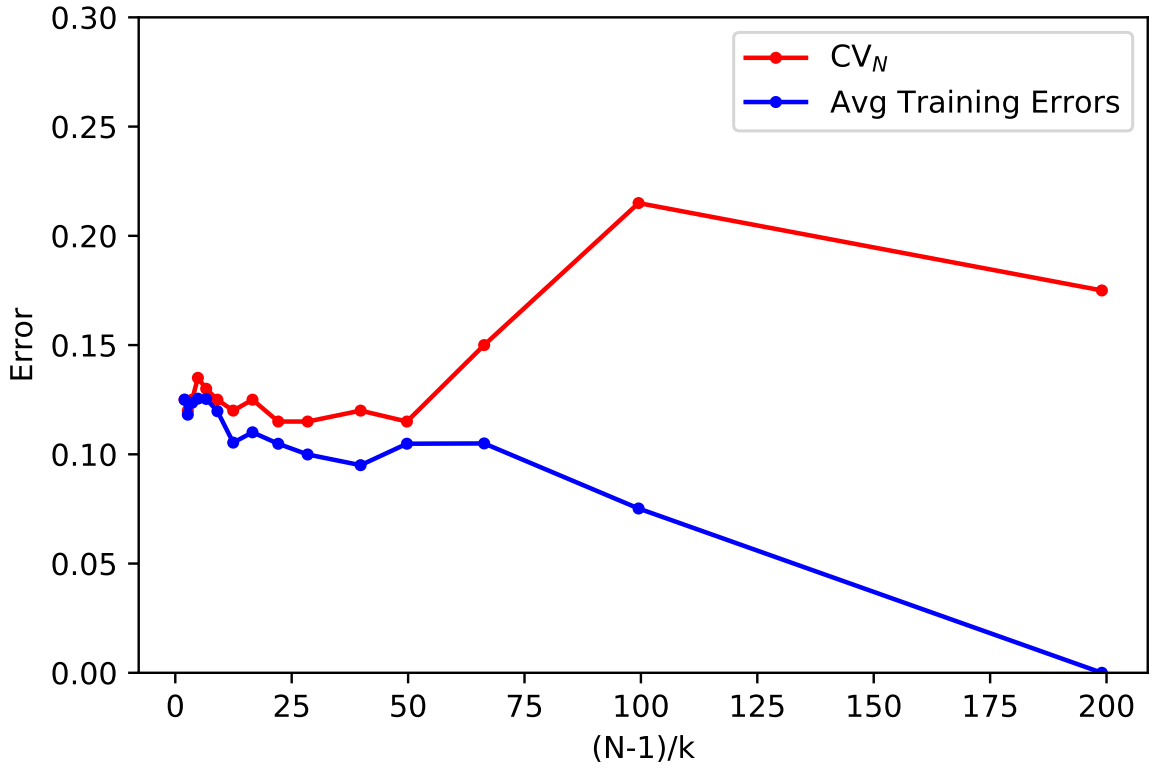
$$\mathrm{CV}_{(N)} = \frac{1}{N} \sum_{i=1}^{N} \mathrm{Err}_i\,. \qquad \text{(for classification)}$$

Returning to our running example,

i.e., classifying 3 Gaussian blobs in $\mathbb{R}^2$ associated with 2 classes, $N = 200$, ...



LOOCV provides this test error estimate:



Conclusions

Pros:

- LOOCV provides a better test error estimate
- Easier to detect optimal parameters (*e.g.*, k in k-NN)

Cons:

- Very time consuming: we must fit the data N times

remark: $\exists$ a simplification for LS that allows a single fit;
but for all other methods, it's *looooong*

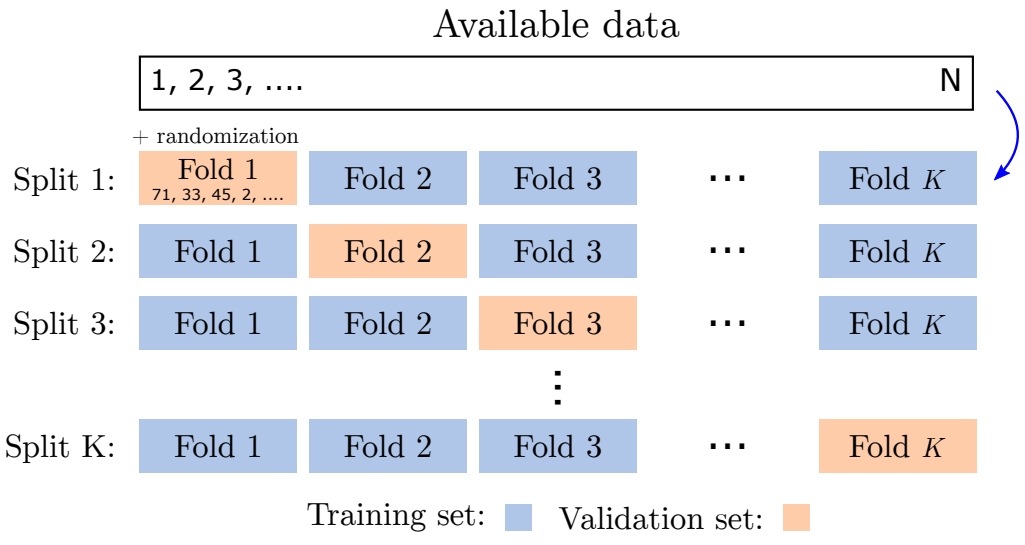
K-Fold Cross-Validation

Fact: LOOCV is not super effective. Why?

- All training sets (from each split) are very similar;
  $\Rightarrow$ only one observation differs.
- Not sure it's worth its computational burden.

Alternative: After the *available/test* data split, use K-Fold Cross-Validation

$\Rightarrow$ Split *available data* in K folds (of size N/K) $\rightarrow$ K possible splits!



Typical value: K set to 5 or 10 (as in Scikit-Learn). (K-Fold CV $\neq$ k-NN)

Mathematically, first, randomized available data, then

1. Create your K folds,

$$\mathrm{Fold}_i \subset [N] = \{1, \dots, N\}, \quad \text{with } |\mathrm{Fold}_i| = \tfrac{N}{K},\ \bigcup_{i=1}^{K} \mathrm{Fold}_i = [N].$$

2. For each split i in $\{1, \dots, K\}$,
   - Fit your model on $\mathcal{T}_i := \{(x_j, y_j) : j \notin \mathrm{Fold}_i\}$ and get $\hat{f}_i$
   - Compute the prediction error on i-th fold $\mathcal{T}_i = \{(x_j, y_j) : j \in \mathrm{Fold}_i\}$

$$\mathrm{MSE}_i := \mathrm{MSE}(\mathcal{T}_i) = \tfrac{K}{N} \textstyle\sum_{j \in \mathrm{Fold}_i} \left(y_j - \hat{f}_i(x_j)\right)^2, \qquad \text{(for regression)}$$

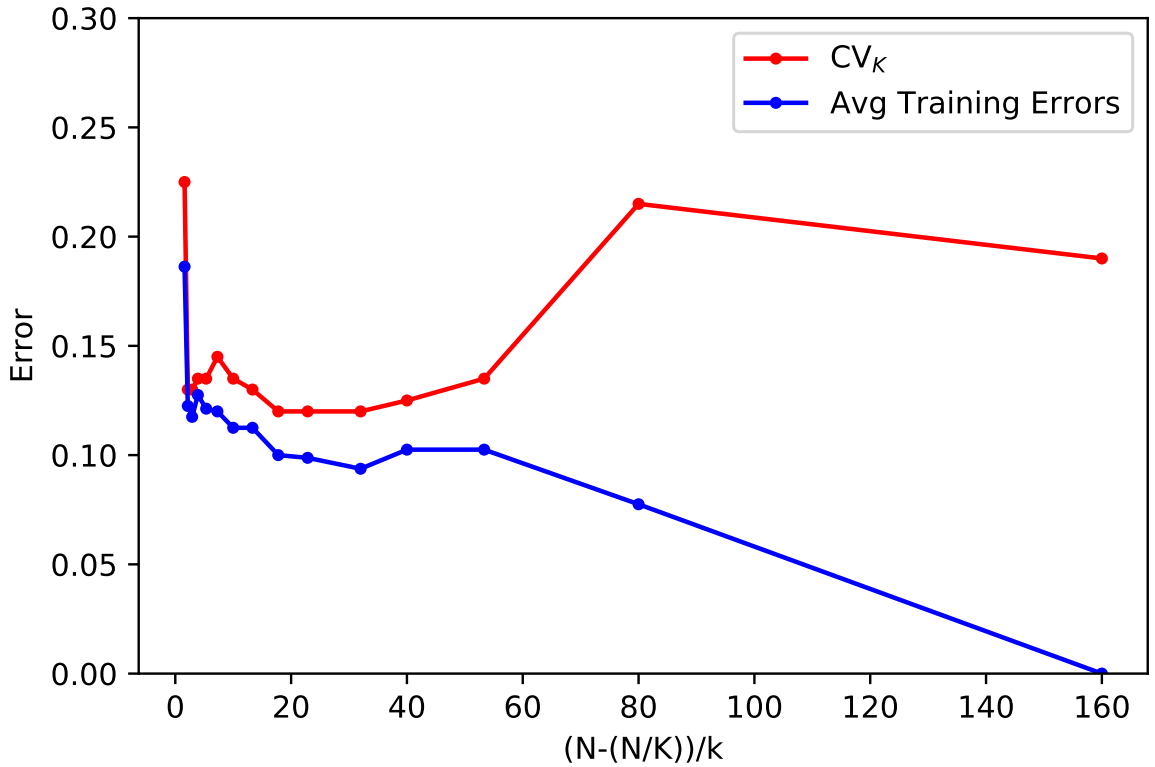
$$\mathrm{Err}_i := \mathrm{Err}(\mathcal{T}_i) = \tfrac{K}{N} \textstyle\sum_{j \in \mathrm{Fold}_i} I\left(y_j \neq \hat{f}_i(x_j)\right). \qquad \text{(for classification)}$$

3. Test error estimate = average on all splits

$$\mathrm{CV}_{(K)} = \tfrac{1}{K} \textstyle\sum_{i=1}^{K} \mathrm{MSE}_i, \qquad \text{(for regression)}$$

$$\mathrm{CV}_{(K)} = \tfrac{1}{K} \textstyle\sum_{i=1}^{K} \mathrm{Err}_i\,. \qquad \text{(for classification)}$$

Returning to our running example, for $K = 10$, we get:



$\Rightarrow$ not very far from LOOCV, but 20 times faster!

2.4. Resampling Methods

2.5. More on Classification Methods

Overview of classification (1/2)

Reminder:

- Classification = predicting a category Y (or G),
  i.e., a *qualitative* (or *categorical*) value
  (*e.g.*, the eye color, a film category)

Examples (from ISL book)

- Given a patient with several observable symptoms,
  what's the diagnosis among K possible diseases?
- We observe DNA sequences of many patients,
  which DNA mutations are *deleterious* (disease causing)?

Overview of classification (2/2)

An algorithm performing classification, or classifier,

- often predicts the probabilities that outcome $\in$ each category;
- from these probabilities, a decision is made
  (*e.g.*, pick category with highest probability).

There exist many classification techniques:

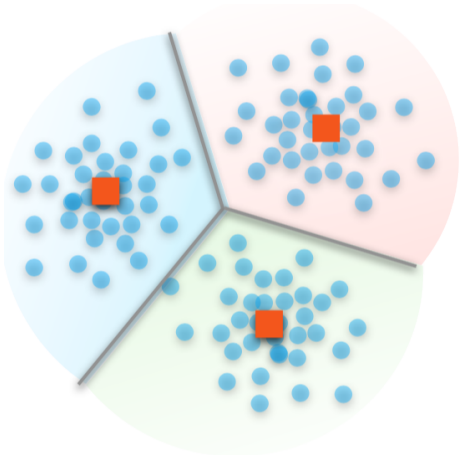
- Linear model (+ LS) + "thresholding" (linear decisions, already covered)
- k-Nearest Neighbors (k-NN) (non-linear dec., already covered)
- Logistic regression (linear decisions, this part)
- Linear Discriminant Analysis (LDA) (linear, not covered)
- Support Vector Machine (SVM) (linear, not covered)
- Quadratic Discriminant Analysis (LDA) (non-linear, not covered)
- Kernel Support Vector Machine (KSVM) (non-linear, not covered)

Note: We key on binary classification, with some insight on multi-class case.

Linear classification methods

Given:

- labels $\mathcal{G} = \{g_1, \cdots, g_Q\}$ (Q classes)
- an input space $\mathbb{R}^p$.



Principle: Classifying by dividing $\mathbb{R}^p$ in Q regions, delineated by linear boundaries

Boundaries: given discriminant functions $\delta_q(x)$, $1 \leqslant q \leqslant Q$,

$$q\text{-th boundary} = \{x : \delta_q(x) = 0\} = \text{a plane}$$

Classification $\equiv$ predict a class from the δ_q's by

- thresholding them (for binary class)
- or sorting them (for multi-class)

Examples:

- Thresholded linear models (*i.e.*, $Q = 1$, $\delta(x) := x^\top \beta - \tau$),
- logistic regression (see later for δ_q), ...

Why not always classify with linear regression? (1/2)

Let's take an example to see it's **not always a good idea**.

Classical industrial quality monitoring:



- A camera observe fruits on a rapid conveyor belt
- We observe an image X in a category $G \in \mathcal{G}$ with $\mathcal{G} = \{'\text{good}', '\text{ nonstandard}', '\text{ rotten}'\}$

Training dataset: $\mathcal{T} = \{(x_i, g_i)\}_{i=1}^{N} \subset \mathbb{R}^p \times \mathcal{G}$, with $(x_i, g_i) \sim (X, G)$

- for each fruit, $x_i \in \mathbb{R}^p$ is an image
  e.g., $p = 256 \times 256$, with 8 bits per pixel (each pixel $\in \{0, \cdots, 255\} \subset \mathbb{R}$)






- $g_i \in \mathcal{G}$ is the fruit quality.

Prediction: Given a new image x, what is its category g?

To do a linear regression, you could consider these two possible codings

$$Y(G) = \begin{cases} 1 & \text{if } G = \text{'good'} \\ 2 & \text{if } G = \text{'nonstandard'} \\ 3 & \text{if } G = \text{'rotten'} \end{cases} \quad \text{or} \quad Y(G) = \begin{cases} 3 & \text{if } G = \text{'good'} \\ 2 & \text{if } G = \text{'nonstandard'} \\ 1 & \text{if } G = \text{'rotten'} \end{cases}$$

and then:

1. apply least squares regression to $\mathcal{T} = \{(x_i, y_i(g_i))\}_{i=1}^N$ and get $\hat{\beta}$,
2. and decide the class by rounding $\hat{Y}(x) := \text{round}(x^\top \hat{\beta}) \in \{1, 2, 3\}$.

How to choose? Is there a good coding?

$\Rightarrow$ No good ordering (except for binary classification)

Was it really ok for binary coding? *e.g.*, for 0-1 coding?

No, interpretability problem: for LS + thresholding: $X^\top \hat{\beta} \notin [0, 1]$!?

Possible solution: use the Logistic Regression

Let's simplify the context: Binary class problem in 1-D

- $Y = G \in \{g_1 = 0, g_2 = 1\}$, $X \in \mathbb{R}$.

Logistic regression solution: Don't model Y directly.

Model the probability that $Y = 1$ (or 0) given $X = x$ (the input), *i.e.*,

$$h(x) := \mathrm{p}(Y = 1 | X = x).$$

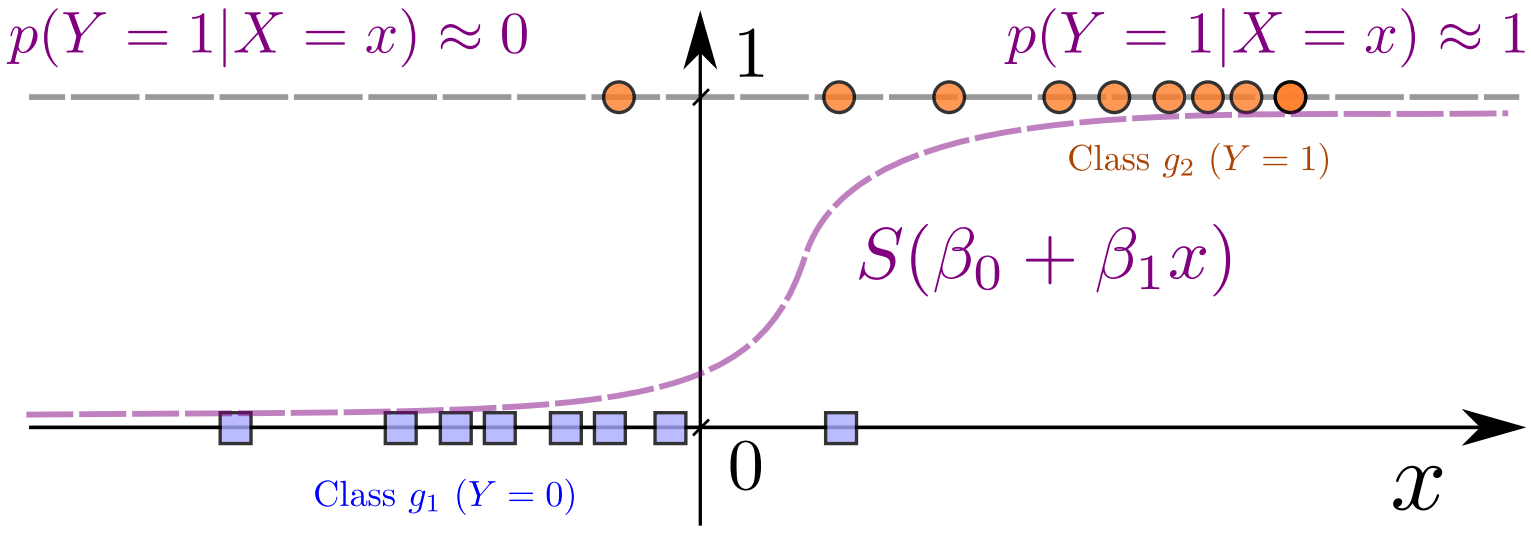
Which model for h?

- Not linear, since $h \in [0, 1]$ and a linear model is unbounded.
- We use the logistic (or logit) function with $\beta = (\beta_0, \beta_1)^\top$, *i.e.*,

$$h(X) = h_\beta(X) := S(\beta_0 + \beta_1 X) := \frac{\exp(\beta_0 + \beta_1 X)}{1 + \exp(\beta_0 + \beta_1 X)}, \quad \left[\begin{array}{c}\text{Generalized}\\ \text{Linear Model}\end{array}\right]$$

  and the sigmoid function $t \in \mathbb{R} \mapsto S(t) = \frac{e^t}{1+e^t} = \frac{1}{1+e^{-t}} \in (0, 1)$.
- As for LR, the *intercept* β_0 and the *slope* β_1 must be optimized

Visually,



Remark: $\mathsf{p}(Y = 0|X) = 1 - h(X) = 1/\big(1 + \exp(\beta_0 + \beta_1 X)\big)$.

The logistic regression is not really a classifier, *i.e.*, it fits $\mathsf{p}(Y|X) \in [0,1]$!

In fact, **Logistic classifier** = Logistic regression + binary conversion.

Given a probability threshold $0 \leqslant \mathsf{p}^* \leqslant 1$,
the estimated class $\hat{G}(x)$ of $x \in \mathbb{R}$ reads

$$\hat{G}(x) = \begin{cases} 1 & \text{if } \mathsf{p}(Y=1|X=x) \geqslant \mathsf{p}^*, \\ 0 & \text{otherwise.} \end{cases} \qquad \text{(binary conversion)}$$

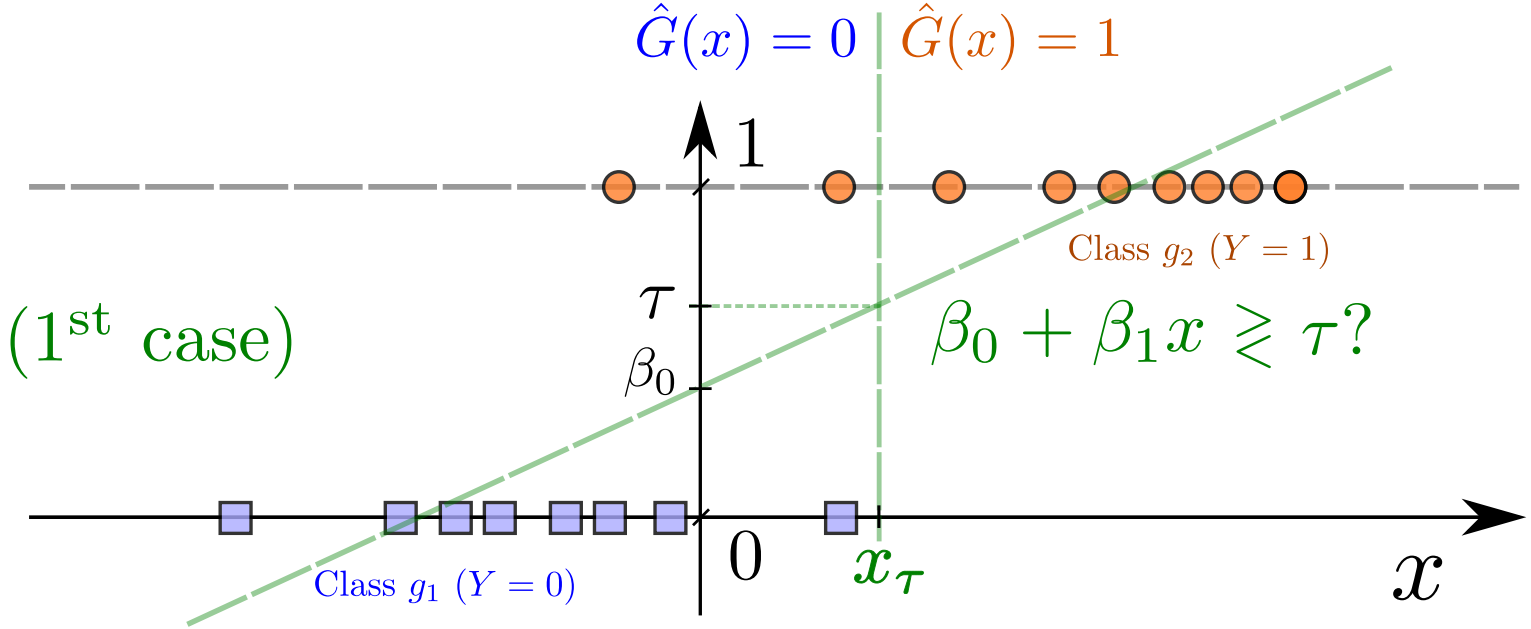
$$= \begin{cases} 1 & \text{if } \log \frac{\mathsf{p}(Y=1|X=x)}{\mathsf{p}(Y=0|X=x)} \geqslant \tau^* := \log \frac{\mathsf{p}^*}{1-\mathsf{p}^*}, \\ 0 & \text{otherwise.} \end{cases} = \begin{cases} 1 & \text{if } \beta_0 + \beta_1 x \geqslant \tau^*, \\ 0 & \text{otherwise.} \end{cases}$$

$\Rightarrow$ $\hat{G}$ splits $\mathbb{R}$ according to sign of $\delta(x) := \beta_0 + \beta_1 x - \tau^*$. (as in binarized LR)

However, we get a specific probabilistic meaning for $\beta_0 + \beta_1 x$!

$$S(\beta_0 + \beta_1 x) = \mathsf{p}(Y=1|X=x) \in [0,1]$$

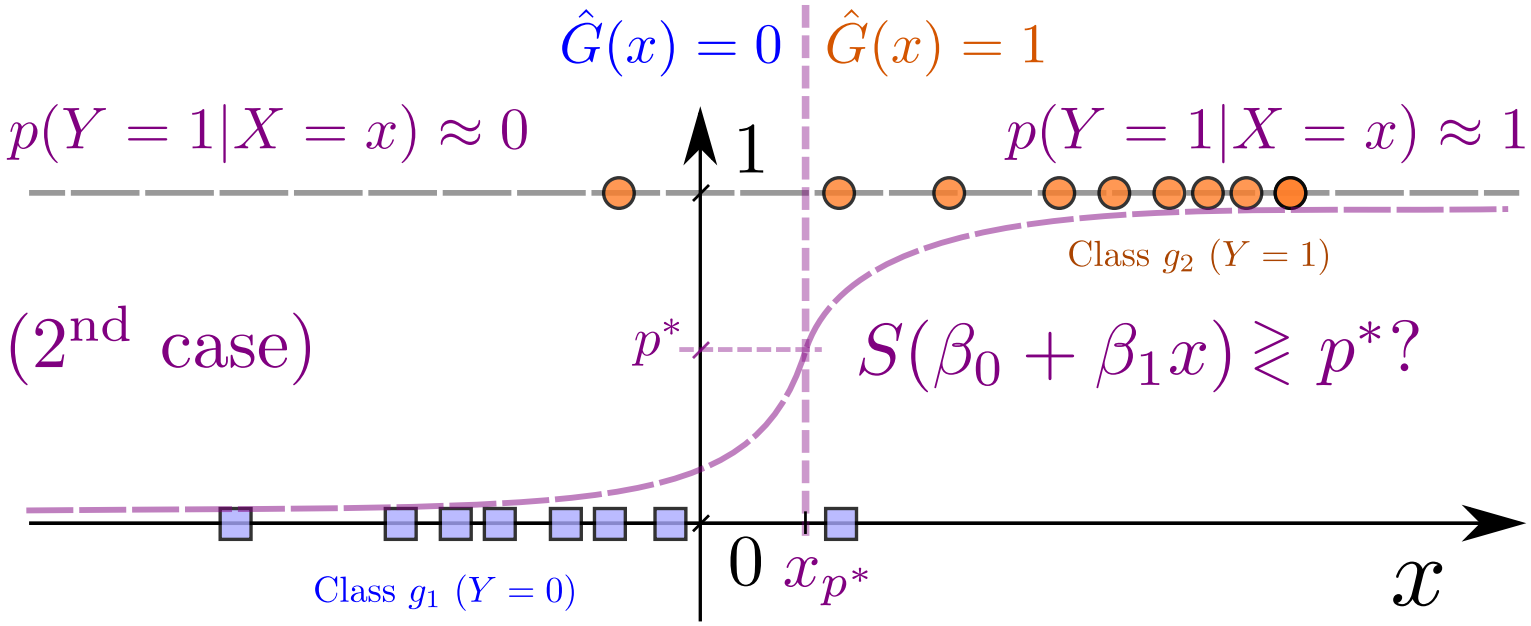
Logistic regression vs Linear regression: visually



Let's consider the classification of two categories $\{g_0 \equiv 0, g_1 \equiv 1\}$.

- **1st case**: Linear regression + binarization (thresholding) with $x_\tau := (\tau - \beta_0)/\beta_1$ set by τ.

Logistic regression vs Linear regression: visually



Let's consider the classification of two categories $\{g_0 \equiv 0, g_1 \equiv 1\}$.

- **1st case**: Linear regression + binarization (thresholding) with $x_\tau := (\tau - \beta_0)/\beta_1$ set by τ.
- **2nd case**: Logistic regression + binarization (thresholding) with $x_{p^*} := (\tau^* - \beta_0)/\beta_1$, and $\tau^* := \log(p^*/(1 - p^*))$ set by p^*.

Reinterpretation: $\exists$ a transformed domain where log. reg. is linear!

Define the logit (or *log-odds*) such that, with $h(X) = S(\beta_0 + \beta_1 X)$,

$$\text{logit}(h(X)) := S^{-1}(h(X)) = \log\left(\frac{h(X)}{1-h(X)}\right) = \log\left(\frac{\mathsf{p}(Y=1|X)}{\mathsf{p}(Y=0|X)}\right) = \beta_0 + \beta_1 X \ \in \mathbb{R}.$$

Therefore,

$$\text{logit}(h(X)) = \begin{cases} \text{strongly positive} & \text{iff "}Y=1\text{" is certain,} \\ \text{strongly negative} & \text{iff "}Y=0\text{" is certain,} \\ 0 & \text{for equipropable "}Y=0\text{" and "}Y=1\text{".} \end{cases}$$

$\Rightarrow$ Logistic regression imposes a linear model on log-odds!

Conclusions:

- Equiprobable classes ($p^* = 1/2$) are split on $x = -\beta_0/\beta_1$;
- sign $\beta_1 \in \{\pm 1\}$ points towards $Y = 1$;
  e.g., if $\beta_1 > 0$, $Y = 1$ is mainly on the right of x_{p^*};
- if $\beta_1 > 0$, an increase of X leads to an increase of $\mathsf{p}(Y = 1|X)$.

Logistic regression with p inputs/dimensions

For p inputs $X_1, \cdots, X_p \in \mathbb{R}$, we define:

- $X = (X_0 = 1, X_1, \cdots, X_p)^\top \in \mathbb{R}^{p+1}$, and $\beta = (\beta_0, \cdots, \beta_p)^\top \in \mathbb{R}^{p+1}$,

so that

$$h(X) = h_\beta(X) := \frac{\exp(\beta_0 + \sum_{i=1}^p \beta_i X_i)}{1 + \exp(\beta_0 + \sum_{i=1}^p \beta_i X_i)} = \frac{\exp(\beta^\top X)}{1 + \exp(\beta^\top X)} = S(\beta^\top X),$$

with β_0 the *intercept* and $(\beta_1, \cdots, \beta_p)^\top$ the *direction* in $\mathbb{R}^p$.

Conclusions: (with $p^* = 1/2$)

- The logistic regression is a linear classification method ($\equiv$ linear boundaries).
- The plane $\mathcal{H} := \{x \in \mathbb{R}^{p+1} : x_0 = 1, \beta^\top x = 0\}$ separates equiprobable classes.
  In fact: $\forall x \in \mathcal{H}$, $\frac{h(x)}{1-h(x)} = \frac{\mathrm{p}(Y=1|x)}{\mathrm{p}(Y=0|x)} = 1 \Rightarrow \mathcal{H} \equiv$ equi-probability plane.
- $(\beta_1, \cdots, \beta_p)^\top$ points towards the *positive* class ($Y = 1$) in $\mathbb{R}^p \rightarrow$ Why ?
- if $\beta_j > 0$ ($j \geqslant 1$), an increase of X_j leads to an increase of $\mathrm{p}(Y = 1|X)$.

Fitting the logistic regression model (1/3)

Fitting is done by maximum likelihood estimation (MLE), *i.e.*,

"*... a method of estimating the parameters of a probability distribution (i.e., β) by maximizing a likelihood function, so that under the assumed statistical model the observed data is most probable.*" (source: Wikipedia)

$\Rightarrow$ Let's find our likelihood function! (for p inputs)

1. $Y|X$ is binary and $\mathrm{p}(Y = 1|X = x) = h_\beta(x) \Rightarrow Y|X \sim \mathrm{Ber}(h_\beta(x))$



2. Under the logistic regression model, likelihood of $Y = y$ given $X = x$:

$$\mathrm{p}(Y = y|X = x) = \begin{cases} h_\beta(x) & \text{if } y = 1, \\ 1 - h_\beta(x) & \text{if } y = 0, \end{cases} = h_\beta(x)^y (1 - h_\beta(x))^{1-y}.$$

3. Given dataset $\mathcal{T} = \{(x_i, y_i)\}_{i=1}^N$, likelihood of $\mathcal{T}$:

$$\begin{aligned} L(y_1, \cdots, y_N|\beta, x_1, \cdots, x_N) &:= \mathrm{p}(Y_1 = y_1, \ldots, Y_N = y_N|X_1 = x_1, \ldots, X_N = x_N) \\ &= \prod_{i=1}^N \mathrm{p}(Y_i = y_i|X_i = x_i) \\ &= \prod_{i=1}^N h_\beta(x_i)^{y_i} (1 - h_\beta(x_i))^{1-y_i}. \end{aligned}$$

⚠ Assuming all (x_i, y_i) are independent (hence the product $\prod$).

Maximizing $L \Leftrightarrow$ Minimizing the negative log-likelihood $\mathcal{L}(\beta)$, *i.e.*,

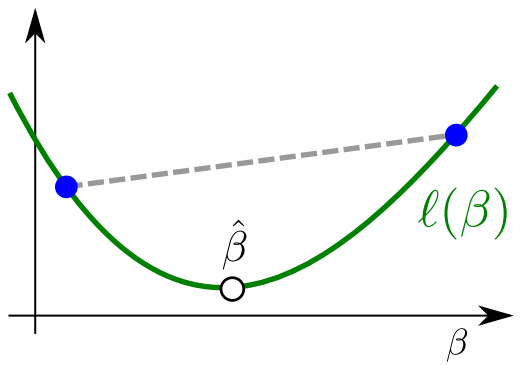
$$\begin{aligned}\mathcal{L}(\beta) = \mathcal{L}(\beta; \mathcal{T}) &:= -\log L(y_1, \cdots, y_N | \beta, x_1, \cdots, x_N) \\ &:= -\left(\sum_{i=1}^N y_i \log h_\beta(x_i) + (1 - y_i) \log(1 - h_\beta(x_i))\right) \\ &:= \sum_{i=1}^N \log(1 + e^{\beta^\top x_i}) - y_i \beta^\top x_i \qquad \text{(each color = convex fct)}\end{aligned}$$

Remarks:

(i) $\mathcal{L} \equiv$ average (binary) *cross entropy*, its loss is the *logistic loss* (or *log* loss).

(ii) $\mathcal{L}$ is convex in β! Addition & composition of convex functions

(or–a bit harder to show–it has a semi-positive definite Hessian).



$\Rightarrow$ One can compute a minimum for $\mathcal{L}$ (convex optimization)

Therefore, optimal logistic parameters $\hat{\beta}$ obtained from

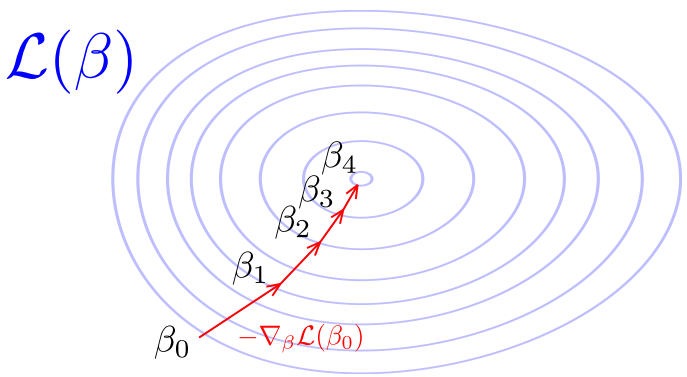
$$\hat{\beta} := \arg\min_{\beta} \mathcal{L}(\beta) = \sum_{i=1}^{N} \log(1 + e^{\beta^\top x_i}) - y_i \beta^\top x_i.$$

One can show that (homework)

$$\nabla_\beta \mathcal{L}(\beta) = \sum_{i=1}^{N} (S(\beta^\top x_i) - y_i) x_i \quad \Rightarrow \text{easy gradient.}$$

$\exists$ many methods to compute $\hat{\beta}$ by minimizing $\mathcal{L}$,

e.g., gradient descent / Newton-Raphson / LBFGS, ...



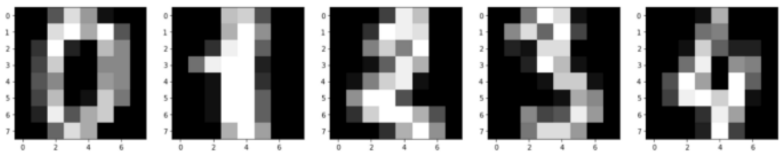
Use your favorite solver, *e.g.*, the methods of scikit learn

A Logistic Regression Example

Let's illustrate the logistic regression with one binary classification example.

Objective: On the digits dataset, create a classifier of "3" vs. "$\neq$ 3".

Dataset: From $\mathcal{T}_{\text{dig}} = \{(x_i, g_i)\}_{i=1}^{N=1797} \subset \mathbb{R}^{p=8^2=64} \times \{\text{'0'}, \cdots, \text{'9'}\}$

digits:  $\cdots$

we create $\mathcal{T} = \{(x_i, y_i)\}_{i=1}^{N} \subset \mathbb{R}^p \times \{\text{'}\neq 3\text{'}, \text{'3'}\} \equiv \mathbb{R}^p \times \{0, 1\}$.

Training/Test splits: Given $\boldsymbol{X} = \{x_i\}_{i=1}^{N}$ and $\boldsymbol{y} = \{y_i\}_{i=1}^{N}$, we split them as

$$\boldsymbol{X} = \boldsymbol{X}_{\text{train}} \cup \boldsymbol{X}_{\text{test}}, \qquad \boldsymbol{y} = \boldsymbol{y}_{\text{train}} \cup \boldsymbol{y}_{\text{test}}.$$

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = \
          train_test_split(X, y, test_size=1/7.0, random_state=0)
```

We can now ask Scikit-Learn to fit a logistic regression on the training data, *i.e.*, to find $\hat{\beta}$ by minimizing $\mathcal{L}(\beta; \boldsymbol{X}_{\text{train}}, \boldsymbol{y}_{\text{train}})$.

This is done as follows:



```
from sklearn.linear_model import LogisticRegression
log_reg = LogisticRegression(solver='lbfgs',penalty='none')
log_reg.fit(X_train, y_train)
```

Scikit-Learn thus learns a classifier:

$$\hat{G} : X \in \mathbb{R}^p \mapsto \hat{G}(X) \in \mathcal{Y} = \{\text{`}\neq 3\text{'}, \text{`}3\text{'}\},$$

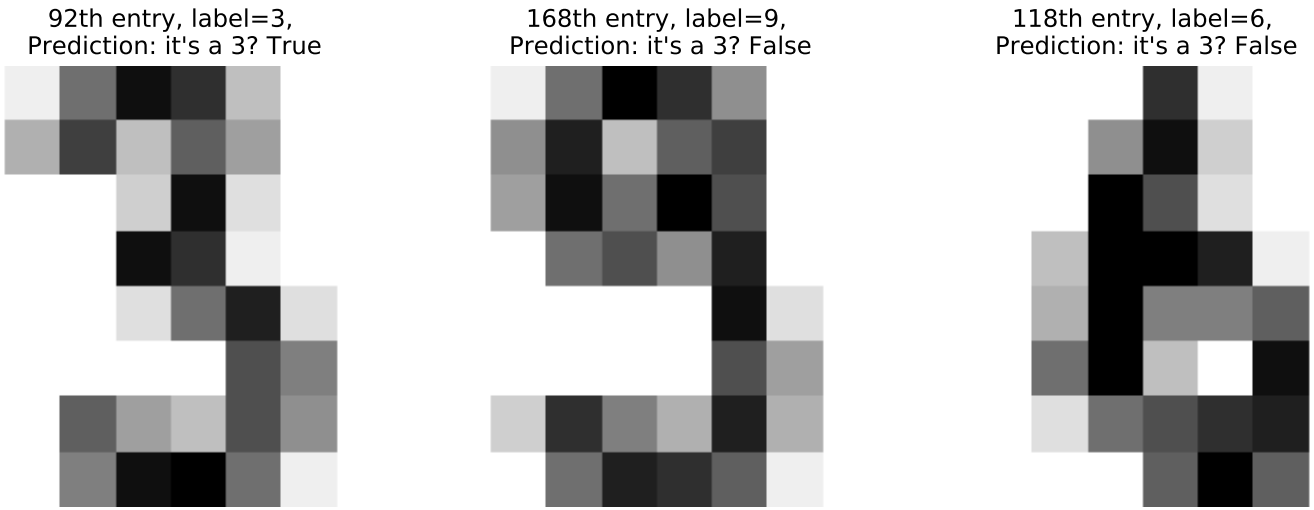
from a logistic regression model, using the training dataset $(\boldsymbol{X}_{\text{train}}, \boldsymbol{y}_{\text{train}})$.

Let's now do some predictions ...

Example of prediction:

```
idx = np.random.randint(np.shape(X_test)[0])
pred_y_idx = log_reg.predict([X_test[idx]])
```

Let's observe 3 instances



Is it a perfect classifier? Performance?

Performance Analysis: Accuracy

We can measure the (empirical) accuracy, or "1 - classification error rate":

$$\mathrm{Acc}^{\mathrm{emp}}(\hat{G}) := \frac{1}{N}\sum_{i=1}^{N} I(y_i = \hat{G}(x_i)) = 1 - \frac{1}{N}\sum_{i=1}^{N} I(y_i \neq \hat{G}(x_i)) = 1 - \mathrm{Err},$$

$\Rightarrow$ (empirical) approximation of $\mathrm{Acc}(\hat{G}) := P(\hat{G}(X) = Y)$.

Methodology: Cross-validation over the training set $\Rightarrow$ a 3-fold CV

```
from sklearn.model_selection import cross_val_score
cross_val_score(log_reg, X_train, y_train, cv=3, scoring='accuracy')
```

Results: `fold 1:` 0.98, `fold 2:` 0.97, `fold 3:` 0.97 $\rightarrow$ Is it good ?

Well, the dumb classifier that always answer "*this is not a 3*" reaches

Results: `fold 1:` 0.89, `fold 2:` 0.90, `fold 3:` 0.90

About 90% accuracy! $\rightarrow$ Explanation: Imbalanced problem!

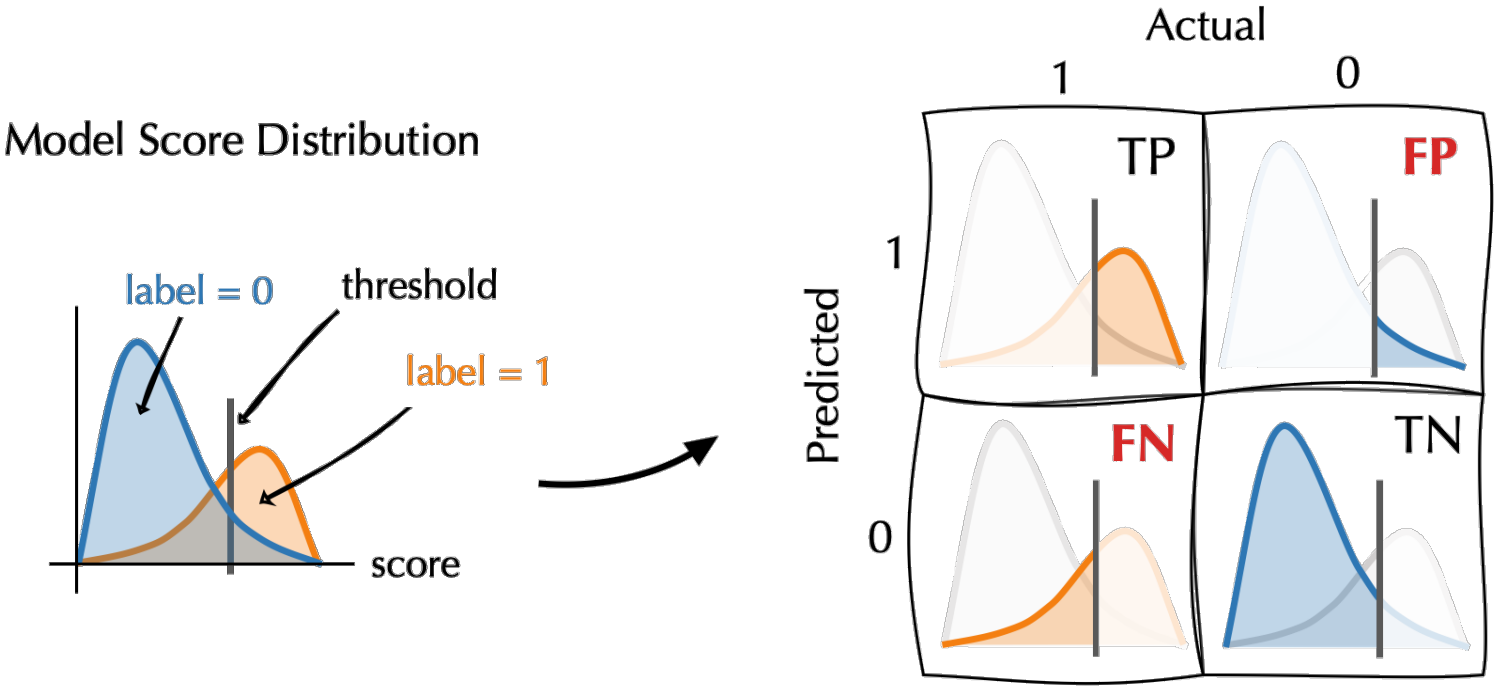
Performance Analysis: TP/TN/FP/FN

Accuracy is not everything! Given '3' $\equiv$ label 1, '$\neq$ 3' $\equiv$ label 0,

and the output of $\hat{G}$, $\rightarrow$ we define the 4 metrics:

- Positive = 1 ('3'),
- Negative = 0 ($\neq$ 3'),

True Positive (TP)	False Positive (FP)
False Negative (FN)	True Negative (TN)



Source: Choosing the Right Model Threshold, by Christopher Bryant

In general, for a Q-class classification problem,

$$2 \times 2 \text{ matrix } \begin{pmatrix} \mathrm{TP} & \mathrm{FP} \\ \mathrm{FN} & \mathrm{TN} \end{pmatrix} \rightarrow Q \times Q \text{ confusion matrix } \mathcal{C} \ ,$$

with $\mathcal{C}_{ij}$ = number of elements of (true) class "j" classified in class "i".

In our binary case ($Q = 2$), we get $\mathcal{C} \in \mathbb{N}^{2\times 2}$:

Decision $\downarrow$ \ Truth $\rightarrow$	"**3**"	"$\neq$ **3**"	$\sum =$
"**3**" ($\rightarrow$ Positive)	True Positive (TP)	False Positive (FP)	$\rightarrow$ all "3" tests (P)
"$\neq$ **3**" ($\rightarrow$ Negative)	False Negative (FN)	True Negative (TN)	$\rightarrow$ all "$\neq$ 3" tests (N)
$\sum =$	$\downarrow$ all "3"s	$\downarrow$ all "$\neq$ 3"s	

How to compute the confusion matrix $\mathcal{C}$? From the training set:

1. Use K-Fold CV, predict all y_i of i-th fold, $1 \leqslant i \leqslant K \Rightarrow$ this gives all $\hat{y}_i$;
2. Use all the true/predicted label pairs $\{(y_i, \hat{y}_i)\}_{i=1}^{N}$ to compute $\mathcal{C}$.

With built-in Scikit-Learn functions, this gives:

```
from sklearn.model_selection import cross_val_predict
from sklearn.metrics import confusion_matrix
# Step 1:
y_train_pred = cross_val_predict(log_reg, X_train, y_train, cv=3)
# Step 2:
C = confusion_matrix(y_train, y_train_pred)
```

Results:

Decision ↓ \ Truth →	"3"	"≠ 3"	∑ =
"3"	148	26	→ 174
"≠ 3"	13	1353	→ 1379
∑ =	↓ 161	↓ 1366	∑ = 1540

In the context of:

- binary classification ($Q = 2$),
- or "binarized" Q-class classification, *e.g.*, {"g", "$\neq g$"} for some $g \in \mathcal{G}$,

confusion matrix summarized to (empirical) precision and recall.

1. **Precision**: (aka *positive predictive value* – PPV) (measure of *quality*)

$$\mathrm{Pre}^{\mathrm{emp}}(\hat{G}) = \frac{TP}{TP+FP} \approx \mathrm{Pre}(\hat{G}) := P(Y = 1 | \hat{G}(X) = 1) = \frac{P(Y=1 \,\&\, \hat{G}(X)=1)}{P(\hat{G}(X)=1)},$$

Interpretation:

- Precision ≡ True positives over Perceived positives
- Out of all the examples the classifier thought were positive (≡ *perceived*), how often were the examples actually positive?

⇒ In our example, we count the number of correctly classified "3"s **over** all the images classified as "3" (*i.e.*, positive test, y=1)

2. Recall: (aka *TPR*, *sensitivity*) (measure of *quantity*)

$$\mathrm{Rec}^{\mathrm{emp}}(\hat{G}) = \frac{TP}{TP+FN} \approx \mathrm{Rec}(\hat{G}) := P(\hat{G}(X) = 1 | Y = 1) = \frac{P(\hat{G}(X)=1 \,\&\, Y=1)}{P(Y=1)}.$$

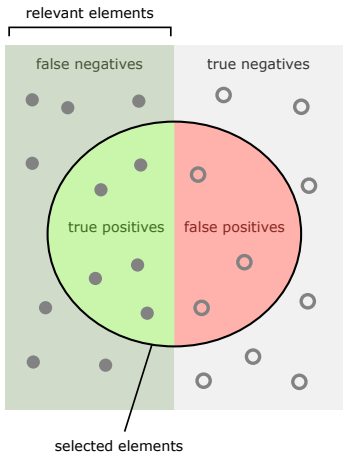
Interpretation:

- Recall ≡ True positives over Real positives
- When presented a real positive example,
  how often does the classifier detect it as positive?

⇒ In our example, we count the number of correctly classified "3"s **over** the number of actual "3"s in the dataset.

Performance Analysis: Precision and Recall (3/3)

Visually,



$\Rightarrow$

How many selected items are relevant?

$$\text{Precision} = \frac{\text{true positives}}{\text{selected elements}}$$

How many relevant items are selected?

$$\text{Recall} = \frac{\text{true positives}}{\text{relevant elements}}$$

Source: Wikipedia, by Walber, CC BY-SA 4.0.

High precision? High recall? What's the best? It depends:

- For Covid-19 test, you may prefer to detect all infected (=1) persons but a few healthy persons are detected positive
  $\Rightarrow$ You want high recall but precision can be lower
- For classifying video content that are safe (=1) for kids you must avoid FP but it's ok if safe video are rejected
  $\Rightarrow$ You want high precision but recall can be lower

Performance Analysis: Precision and Recall

In our running example, we find (from $\mathcal{C}$ computed with 3-fold CV):

$$\text{Precision} = 85\%, \text{ Recall} = 92\%$$

Remarks:

- Taking the *harmonic mean* of precision and recall, we get the F_1 score, *i.e.*,

$$F_1 = \frac{2}{\frac{1}{\text{precision}} + \frac{1}{\text{recall}}} \in [0, 1],$$

  $\Rightarrow F_1 \approx 1$ promotes classifier with *precision* $\approx$ *recall*.

- Our previous "*I deny all 3s*" classifier gets precision = recall = 0%, since $\mathrm{TP} = 0$ and its confusion matrix reads

Decision $\downarrow$ \ Truth $\rightarrow$	"3"	"$\neq$ 3"	$\sum =$
"3"	0	0	0
"$\neq$ 3"	161	1379	1540
$\sum =$	161	1366	$\sum = 1540$

Precision & recall depend on an implicit decision threshold.

Earlier, for logistic regression, we decided that

$$\hat{y}_i = 1 \quad \Leftrightarrow \quad h(x_i) := \mathrm{p}(Y = 1 | X = x_i) \geqslant \mathrm{p}^*.$$

or, equivalently, or $p^* = 1/2$,

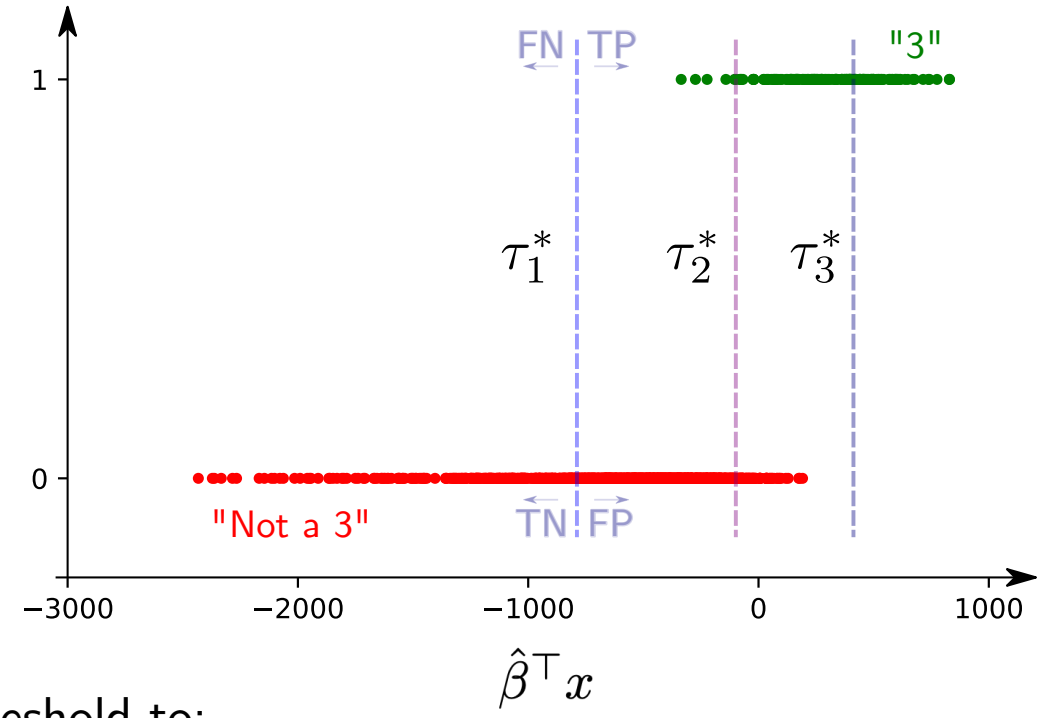
$$\hat{y}_i = 1 \quad \Leftrightarrow \quad \frac{h(x_i)}{1 - h(x_i)} \geqslant \frac{p^*}{1 - p^*} = 1.$$

and, for the logistic model, as soon we get $\hat{\beta}$ from the training/fitting,

$$\hat{y}_i = 1 \quad \Leftrightarrow \quad \log \frac{h(x_i)}{1 - h(x_i)} = \hat{\beta}^\top x_i \quad > \quad \tau^* := \log \frac{p^*}{1 - p^*} = 0.$$

Question: What is the best threshold $\tau^* \in \mathbb{R}$ (*i.e.*, p^*),
that is, the one optimizing a good precision/recall tradeoff ?

This can be seen by plotting all $\{(\hat{\beta}^\top x_i, y_i)\}_{i=1}^N$: (similar to p. 28)



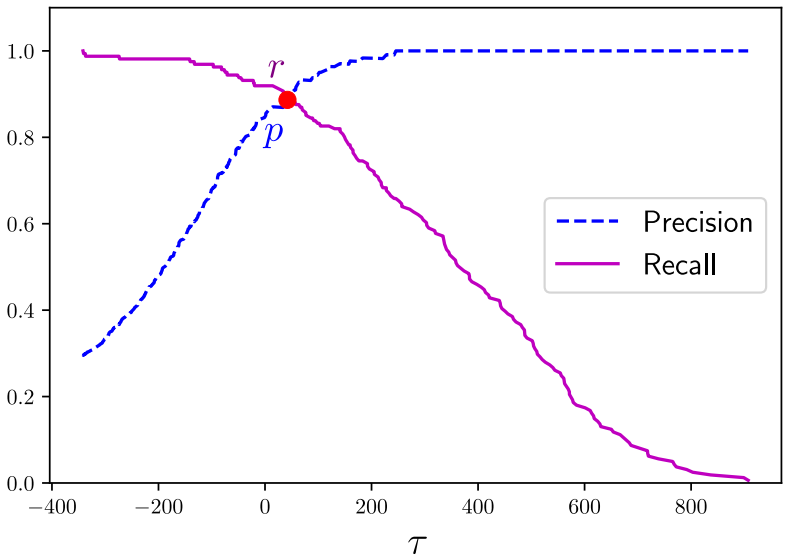
Setting the threshold to:

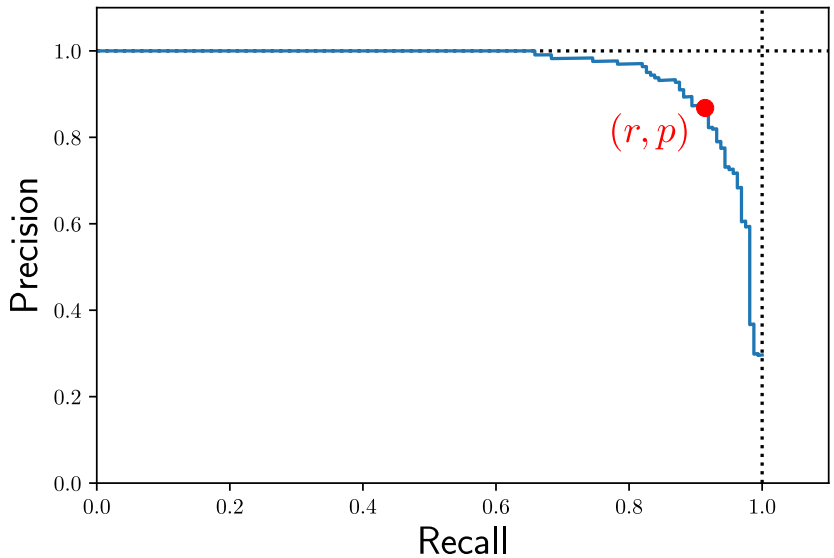
- τ_1^* → all "3"'s are correctly classified (high TP); no FN, but many FPs
  ⇒ low precision, high recall
- τ_3^* → all "$\neq$ 3"s are correctly classified (high TN); no FP, many FNs
  ⇒ high precision, low recall
- τ_2^* → it's in between

Precision and recall tradeoff (3/3)

Let's observe, for our "3" *vs* "$\neq$ 3" classification,

- the *precision* and *recall* as a function of τ on the same plot (left)
- or the precision *vs.* recall, or "PR curve"





$\Rightarrow$ We cannot have together *precision* $\approx$ *1* and *recall* ≈ 1

A tradeoff must be found, depending on your task (*e.g.*, taking $\tau^* = 0$)

Receiver Operating Curve (ROC)

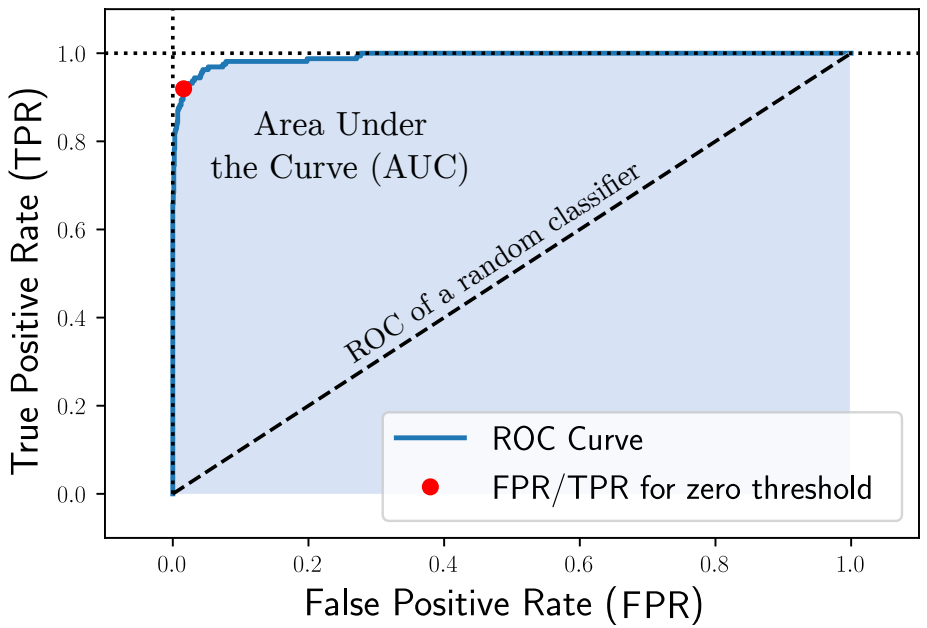
An alternative representation: the Receiver Operating Curve (ROC)

The ROC is the plot of TPR (= recall) *vs.* FPR (aka $1 - \textit{specificity}$) with:

$$\text{TPR} := \frac{TP}{P} = \frac{TP}{TP + FN}, \quad \text{FPR} := \frac{FP}{N} = \frac{FP}{FP + TN}$$

Good classifier
$\Leftrightarrow$ AUC ≈ 1

Random classifier
$\Leftrightarrow$ AUC ≈ 0.5



PR or ROC curves?

- PR: rare "P"s, unbalanced classes, or FPs matter more than FNs
- ROC: better otherwise